This header is used by clients to indicate that a request is coming from an `XMLHttpRequest` object. The server receiving the request can look for the X-Requested-With header to determine where the request is coming from. JQuery always sets this header when making same-origin requests, but removes it from cross-origin requests. The reason is that setting custom request headers requires an additional server configuration step. Rather than force developers to make changes to their server, jQuery chose to drop this header. The following code snippet shows how to reenable the X-Requested-With header on cross-domain requests:

```
$.ajax(url, {
  headers: {'X-Requested-With': 'XMLHttpRequest'}
});
```

Note that if you add this header, you'll also need to update your server to allow this header on cross-origin requests. Chapter 4 shows how to add server-side support for this and other request headers.

2.7 Summary

This chapter explored ways to make cross-origin requests from the browser. Browsers can be divided into three tiers of CORS support:

- Tier 1—Browsers that fully support CORS
- Tier 2—Browsers that partially support CORS
- Tier 3—Browsers that don't support CORS

The browsers in tier 1 all use the `XMLHttpRequest` object to make cross-origin requests, while the browsers in tier 2 (Internet Explorer 8 and Internet Explorer 9) use the `XDomainRequest` object. JQuery can also be used to make cross-origin requests in tier 1 browsers.

In addition to these objects, the browser also uses CORS when loading cross-origin images in the `<canvas>` element. By default, a cross-origin image will taint a canvas, which prevents data from being extracted from the canvas. Setting the image's `crossOrigin` property to `anonymous` (or in some cases `user-credentials`) will allow data to be extracted.

This chapter gives you the foundation for issuing cross-origin requests from the browser. But it has glossed over the details of how the server responds to cross-origin requests. For a cross-origin request to succeed, the server must give its permission to make cross-origin requests. The next part will cover how to configure a server to support CORS. We'll start by learning how to identify and respond to simple CORS requests.